SPRING FRAMEWORK

Aspect-Oriented Programming (AOP)

Introduction • Cross-Cutting Concerns • Scattering & Tangling • Decorator Pattern

---

Study notes based on the provided lecture

Core idea
Separate business logic from supporting infrastructure logic, so business methods remain focused and readable.

1. What Is Aspect-Oriented Programming (AOP)?

AOP stands for **Aspect-Oriented Programming**. It is a separate module in the Spring Framework that helps separate cross-cutting concerns from the main business logic.

AOP can apply supporting logic around method execution:

Before a method executes
↓
During method execution
↓
After a method executes

Example: whenever **createStudent()** is called, the application may need to log the method name, check permissions, calculate execution time, perform auditing, or cache a result. AOP provides a way to separate these operations from the business method.

2. Filters vs Interceptors vs AOP

Concept	What it wraps	Layer
Filter	Raw incoming HTTP request / response	Servlet layer
Interceptor	Controller / handler execution	Spring MVC layer
AOP	Methods of Spring-managed classes	Application method level

Remember: Filters work before DispatcherServlet, Interceptors work around a mapped Spring MVC handler, and AOP can wrap method execution in Spring-managed components.

3. Business Logic vs Infrastructure Logic

Business logic defines what the application does. It is domain-specific.

• Creating a student.

• Managing payments.

• Placing orders.

• Verifying an order.

Infrastructure / supporting logic supports business operations and is often reusable across applications.

• Logging and auditing.

• Security, authentication and authorization.

• Execution-time measurement.

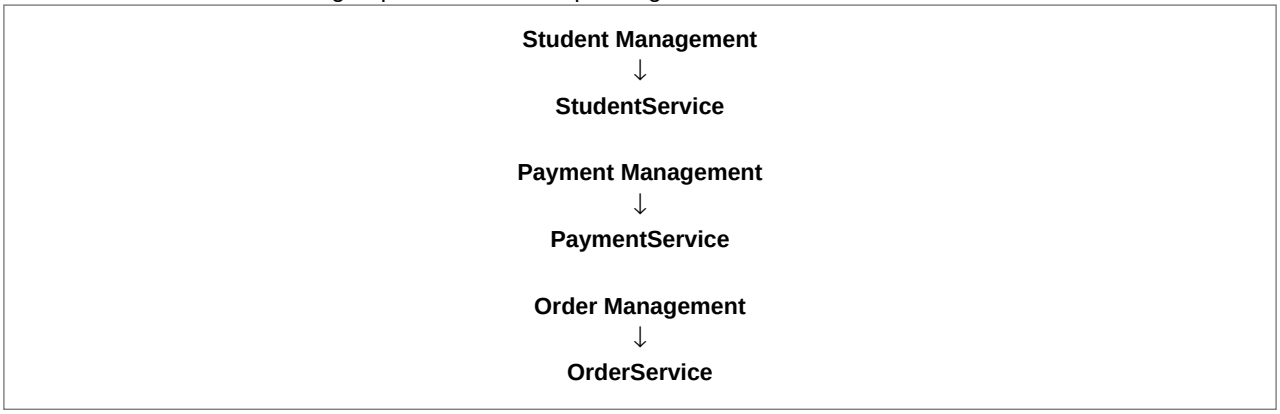
• Caching and exception-related handling.

Business logic changes according to the domain. Infrastructure logic is often similar across different applications, although the exact data or configuration may vary.

4. What Is a Concern?

A **concern** is an area of interest in an application. Examples of business concerns include Student Management, Payment Management and Order Management.

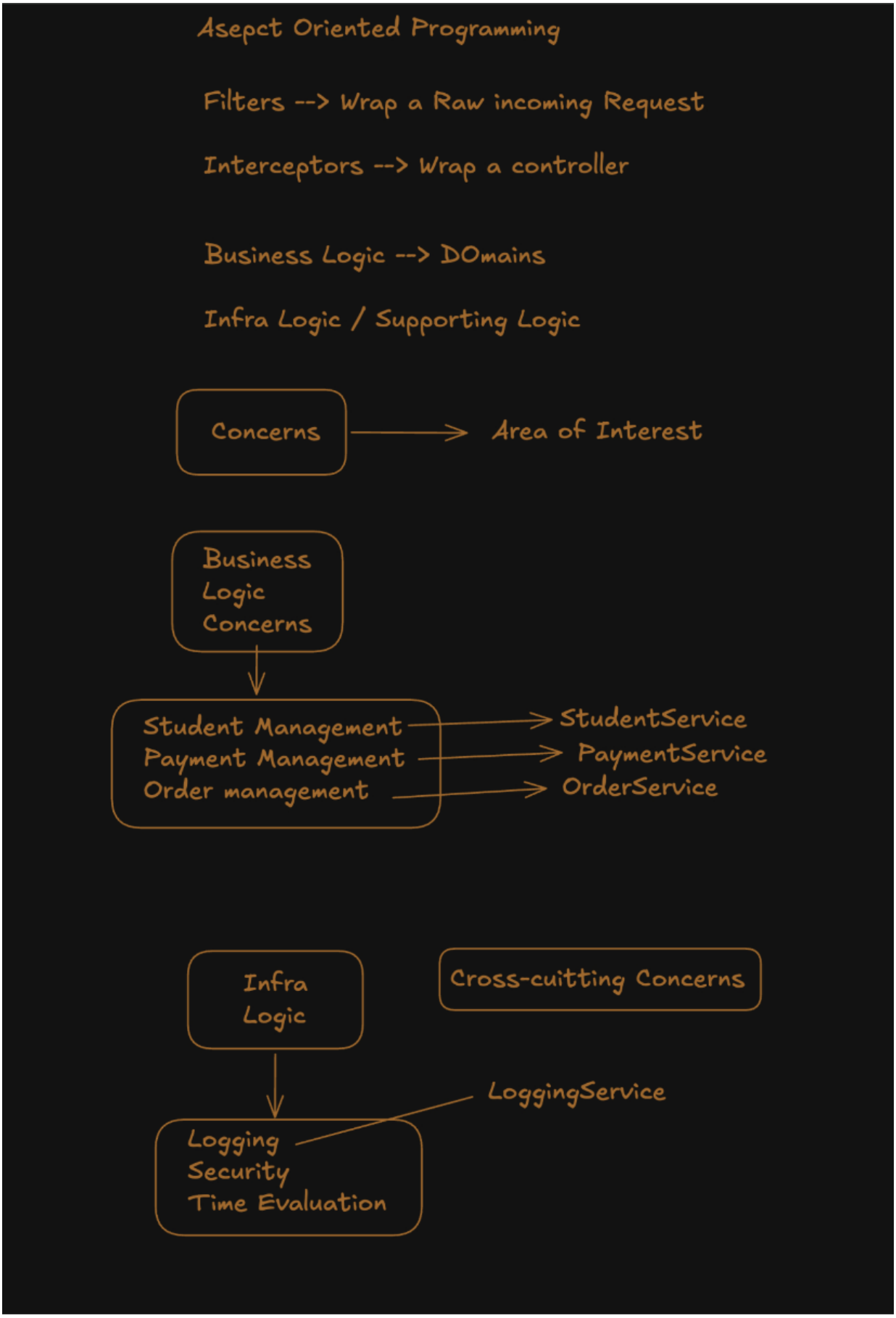
Each business concern can be grouped into its corresponding service:



The methods related to a particular business area are grouped in the service responsible for that area.

Visual Guide 1 — Business Concerns and Cross-Cutting Concerns

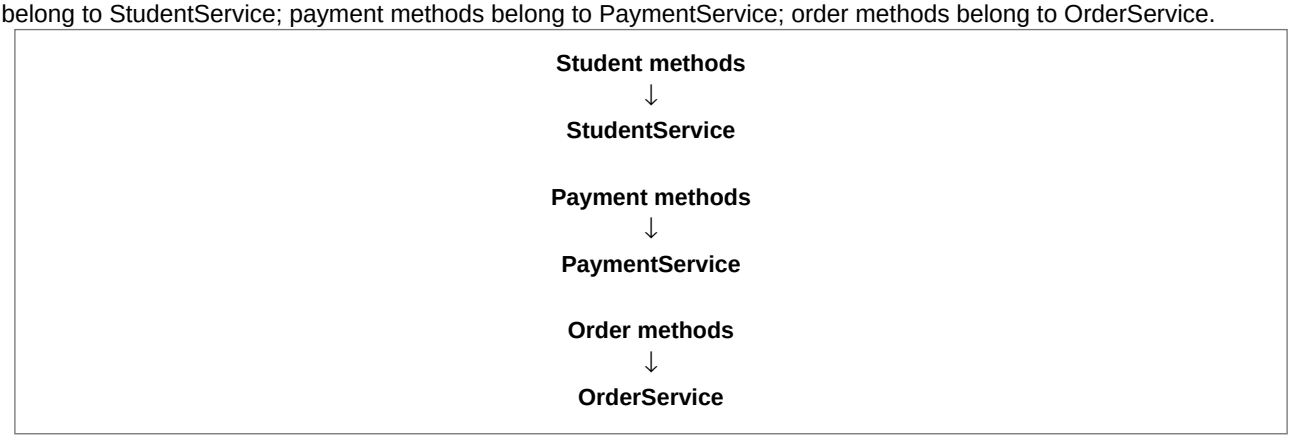
Read this diagram alongside the sections on concerns and business versus infrastructure logic. The upper part distinguishes Filters and Interceptors from AOP. The middle groups domain concerns (student, payment and order management) into their services. The lower part shows infrastructure concerns such as logging, security and time evaluation as cross-cutting concerns.



5. Vertical Concerns vs Horizontal Concerns

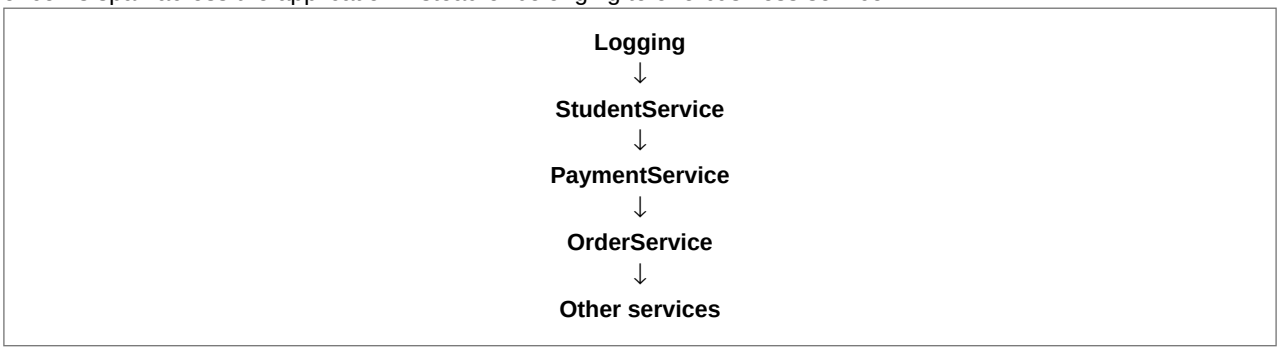
Business logic — vertical concerns

Business methods for one domain are generally grouped inside the corresponding service class. Student methods belong to StudentService; payment methods belong to PaymentService; order methods belong to OrderService.



Infrastructure logic — horizontal concerns

Logging, security and execution-time measurement may be required across many classes and methods. These concerns span across the application instead of belonging to one business service.



6. Cross-Cutting Concerns

Cross-cutting concerns are supporting concerns that affect multiple classes and methods throughout an application.

• Logging.

• Security.

• Execution-time measurement.

• Auditing.

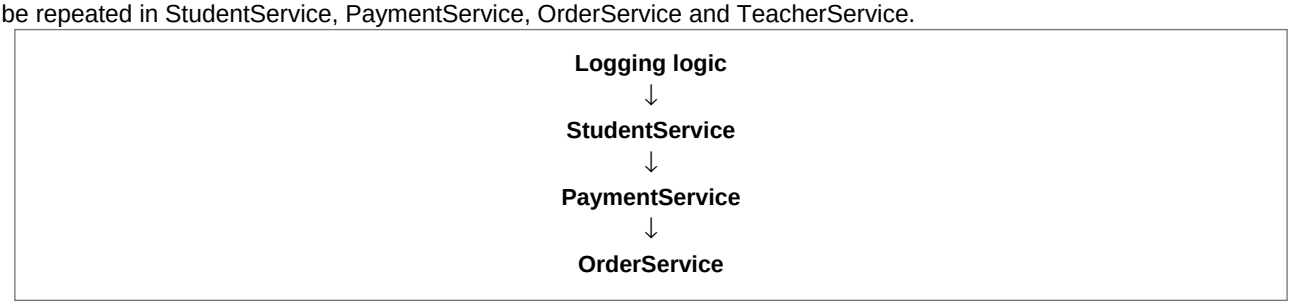
• Caching.

Key point: Business concerns are generally vertical, whereas infrastructure concerns are cross-cutting and span horizontally.

7. Scattering and Tangling

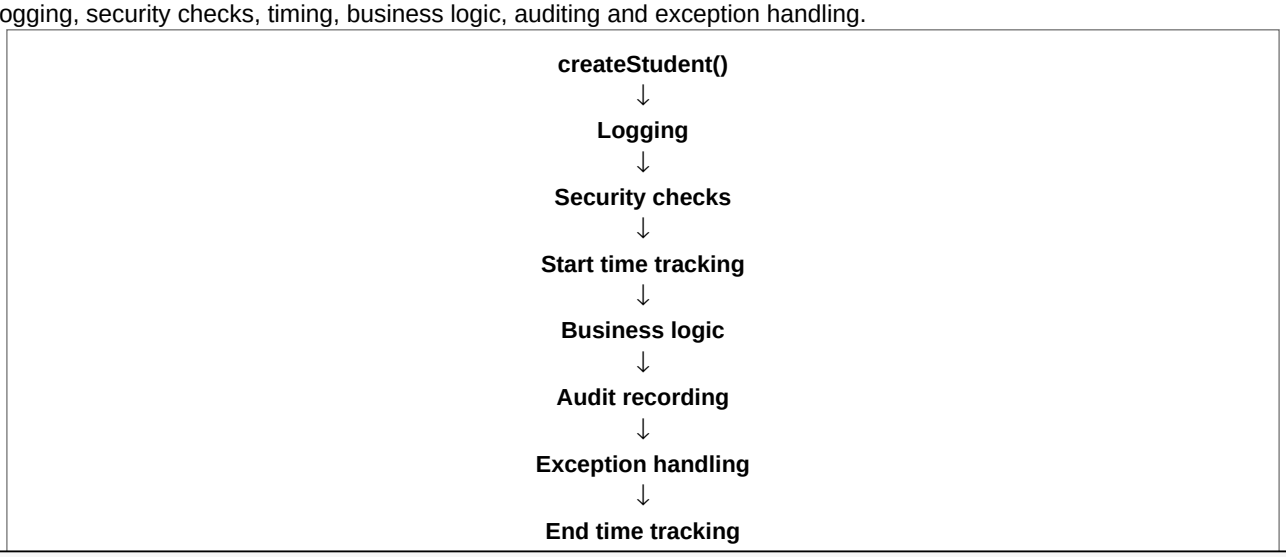
Scattering

Scattering occurs when the same infrastructure logic is spread across multiple classes. For example, logging code may be repeated in StudentService, PaymentService, OrderService and TeacherService.



Tangling

Tangling occurs when multiple types of logic are mixed inside a single method. A createStudent() method may contain logging, security checks, timing, business logic, auditing and exception handling.



Scattering is a problem across classes; tangling is a problem inside a method.

8. Example of a Tangled Business Method

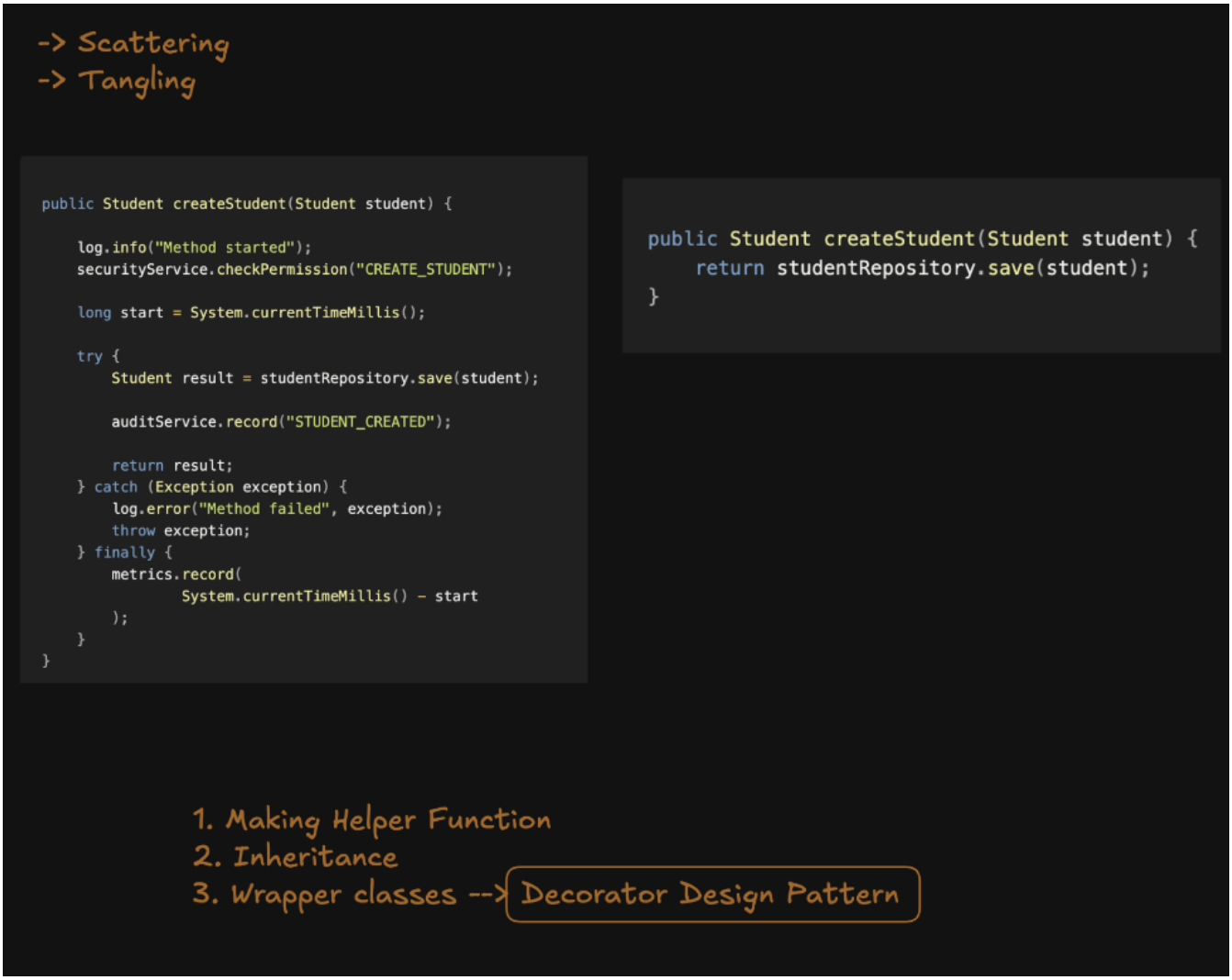
A production createStudent() method may need to log execution, check permissions, measure time, save the student, record an audit event, handle exceptions and record metrics.

```
public Student createStudent(Student student) { log.info("Method started");
securityService.checkPermission("CREATE_STUDENT"); long start = System.currentTimeMillis(); try {
Student result = studentRepository.save(student); auditService.record("STUDENT_CREATED"); return
result; } catch (Exception exception) { log.error("Method failed", exception); throw exception; }
finally { metrics.record(System.currentTimeMillis() - start); } }
```

The actual business operation is only **studentRepository.save(student)**. The remaining code is supporting logic.

Visual Guide 2 — Scattering, Tangling and the Clean Business Method

The left code sample mixes logging, permission checks, timing, business persistence, auditing and exception handling in one method. This illustrates tangling. The simpler method on the right contains only the student save operation. The notes explain why helper functions and inheritance still require explicit calls, and introduce wrappers as another approach.



Key point: scattering means the same supporting concern is repeated across multiple classes; tangling means different concerns are mixed inside one method.

9. What Should the Ideal Method Look Like?

Ideally, the business method should contain only its actual business responsibility.

```
public Student createStudent(Student student) { return studentRepository.save(student); }
```

This makes the method easier to read, understand, maintain and test. However, production applications still need supporting operations such as logging, security, auditing and timing.

10. First Approach — Helper / Utility Functions

A separate utility class can centralize logging methods. Business methods can call those methods explicitly.

```
public class LoggingServiceUtil { public static void logStart(String className, String methodName) {
System.out.println("Executing " + className + ":" + methodName); } public static void logEnd(String
className, String methodName) { System.out.println("Finishing " + className + ":" + methodName); } }
```

Why this is insufficient:

• Business methods still need explicit helper calls.

• Developers may forget to call a helper.

• Exception handling adds more complexity.

• The business method remains mixed with infrastructure calls.

Creating a helper class moves the supporting code, but does not remove the explicit calls from business methods.

11. Second Approach — Inheritance

A parent class can contain common infrastructure methods, and service classes can extend it.

```
public class LoggingService { public void logStart() { } public void logEnd() { } } public class
StudentService extends LoggingService { public Student createStudent(Student student) { logStart(); //
Business logic logEnd(); return student; } }
```

Why inheritance is insufficient:

• The child class still needs to call the inherited methods.

• Infrastructure calls remain inside the business method.

Both helper functions and inheritance still require explicit calls from business methods.

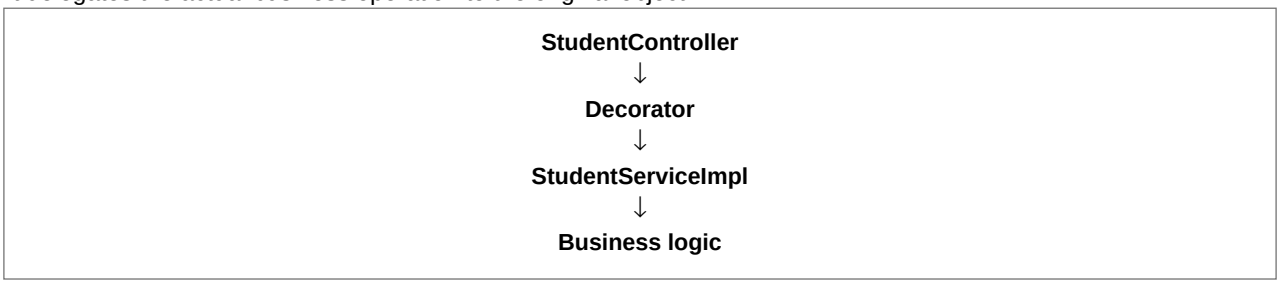
12. Third Approach — Wrapper Classes (Decorator Pattern)

The lecture introduces the **Decorator Design Pattern** as a way to wrap the business object instead of placing infrastructure logic directly inside it.

• The wrapper behaves like the original business object.

• It performs infrastructure logic before and/or after the business operation.

• It delegates the actual business operation to the original object.



13. Implementing the Decorator Pattern

Step 1 — Create an interface

```
public interface StudentService { void createStudent(Student student); }
```

Step 2 — Create the main implementation

```
@Service public class StudentServiceImpl implements StudentService { @Override public void
createStudent(Student student) { // Actual business logic System.out.println("Student Saved"); } }
```

Step 3 — Create the decorator

```
@Component @Primary public class LoggingDecorator implements StudentService { private final
StudentServiceImpl studentServiceImpl; public LoggingDecorator(StudentServiceImpl studentServiceImpl)
{ this.studentServiceImpl = studentServiceImpl; } @Override public void createStudent(Student student)
{ System.out.println("Executing createStudent"); studentServiceImpl.createStudent(student);
System.out.println("Finishing createStudent"); } }
```

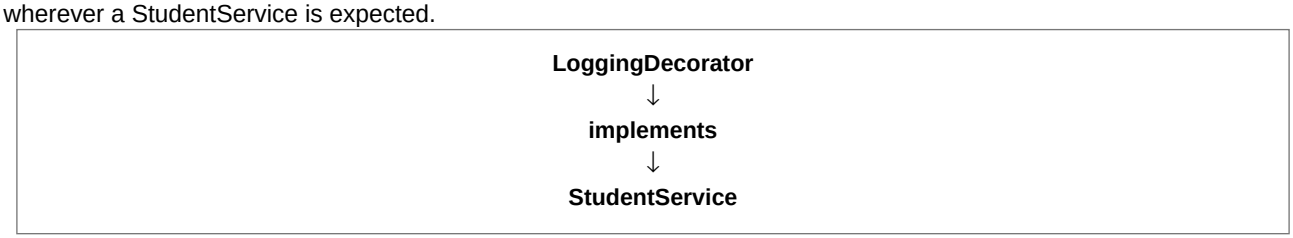
Step 4 — Inject the interface into the controller

```
public StudentController(StudentService studentService) { this.studentService = studentService; }
```

14. Is-a and Has-a Relationships

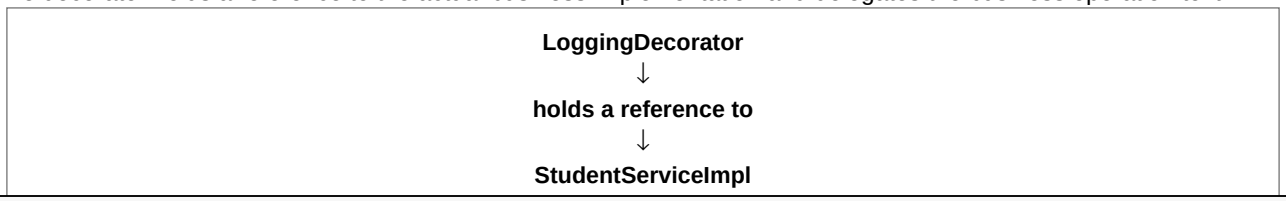
Is-a relationship

The decorator implements the same StudentService interface as the original implementation. Therefore, it can be used wherever a StudentService is expected.



Has-a relationship

The decorator holds a reference to the actual business implementation and delegates the business operation to it.



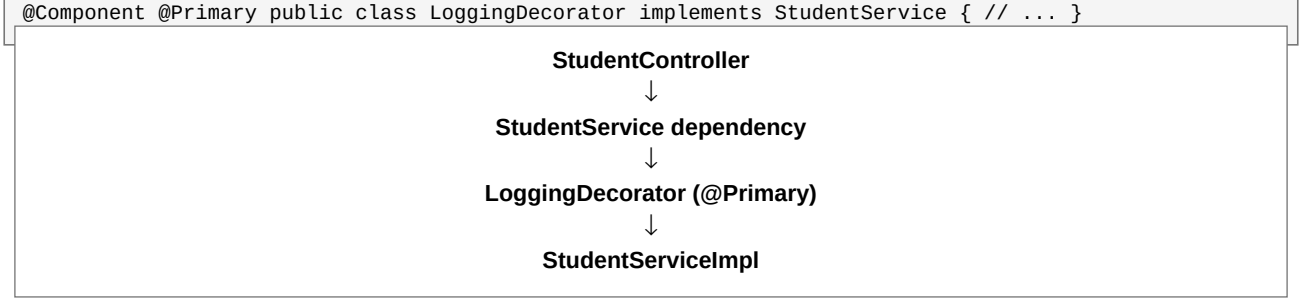
Memory trick: Is-a = implements the same interface. Has-a = holds the original implementation as a dependency.

15. Why Use @Primary?

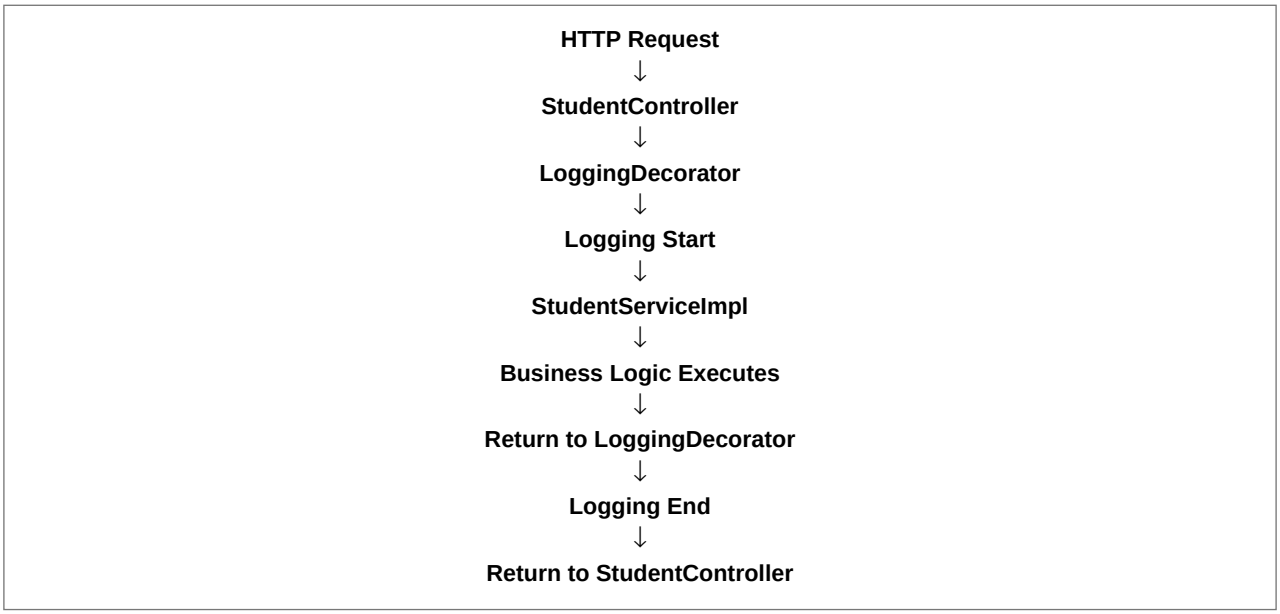
After creating the interface, two Spring beans implement StudentService: StudentServiceImpl and LoggingDecorator. When the controller requests StudentService, Spring needs to know which bean to inject.

Adding **@Primary** to LoggingDecorator tells Spring to prefer it when multiple beans of the same type are available for autowiring.

```
@Component @Primary public class LoggingDecorator implements StudentService { // ... }
```

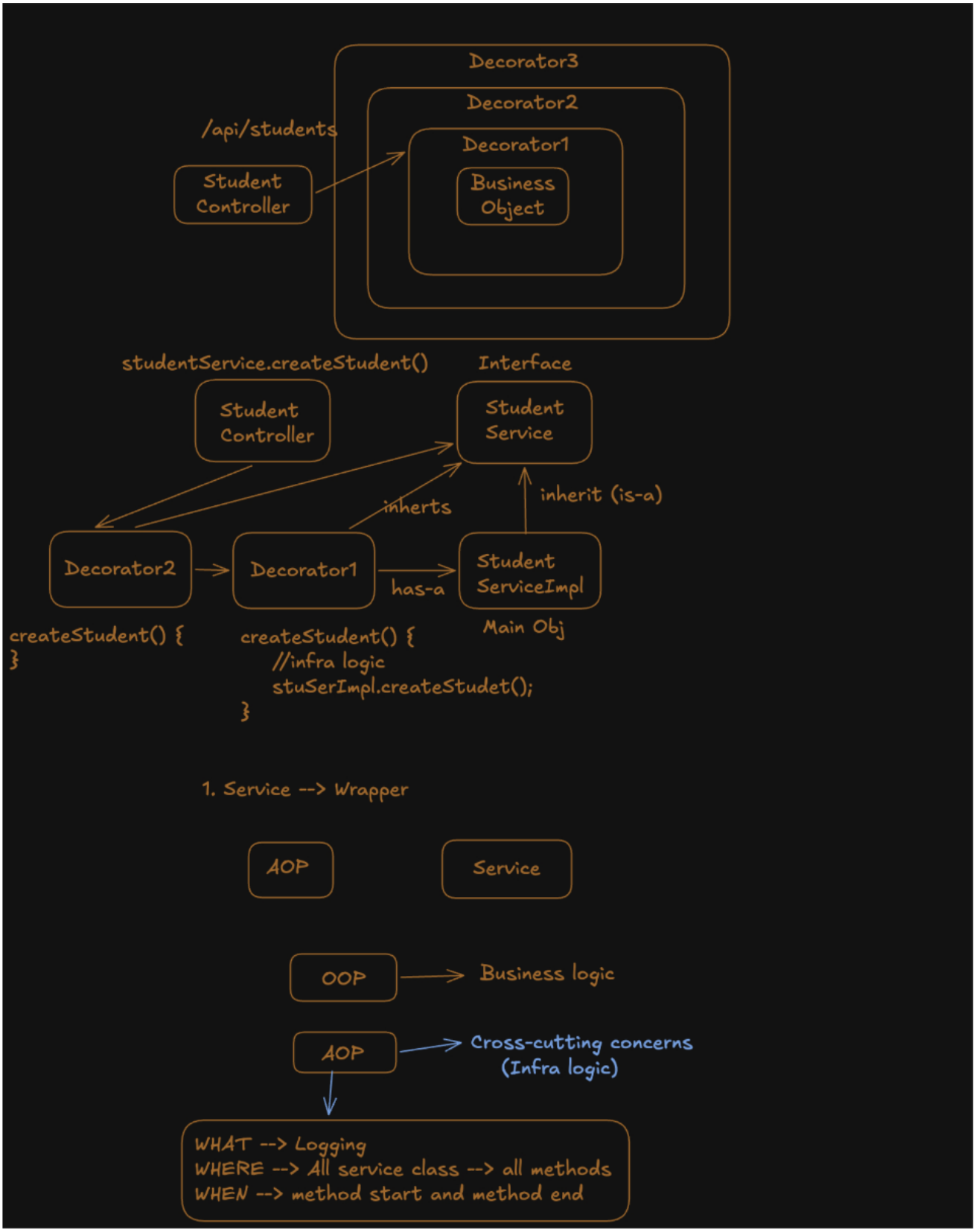


16. Actual Execution Flow



Visual Guide 3 — Decorator Pattern and the AOP Connection

The upper diagram shows nested decorators surrounding the business object. The middle diagram shows the interface relationship (is-a) and delegation relationship (has-a). The lower part contrasts OOP's focus on business logic with AOP's focus on cross-cutting infrastructure concerns, then summarizes the WHAT, WHERE and WHEN questions.



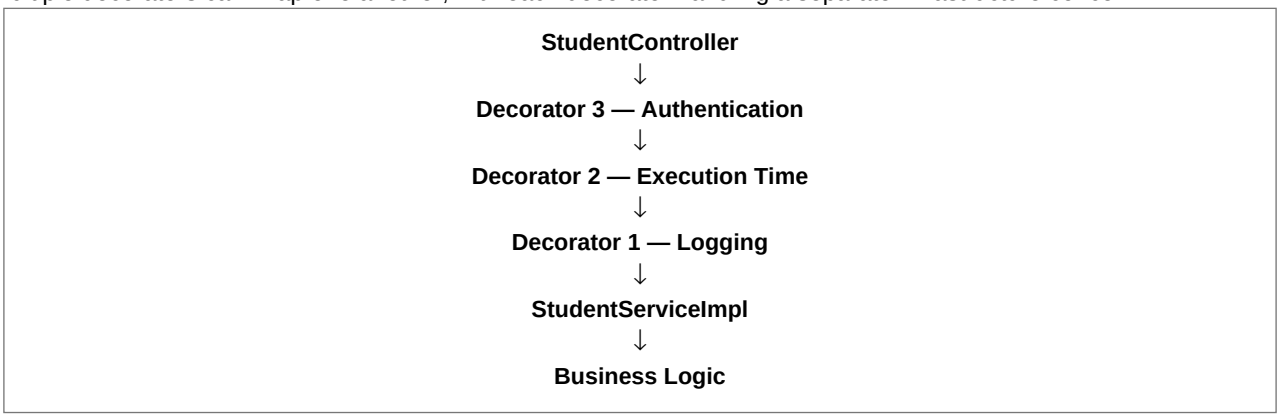
Use this visual when revising how a decorator can add supporting behavior around a service while delegating the actual business operation to the underlying implementation.

↓

HTTP Response

17. Multiple Decorators

Multiple decorators can wrap one another, with each decorator handling a separate infrastructure concern.



The call moves through the decorators before reaching the actual business implementation. When the business operation finishes, control returns through the decorators.

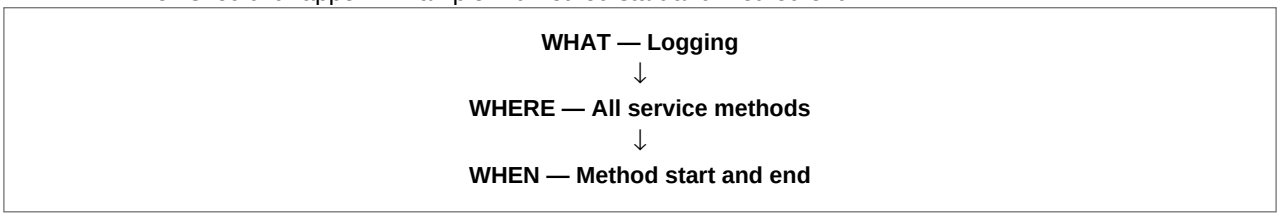
18. OOP vs AOP

OOP	AOP
Object-Oriented Programming	Aspect-Oriented Programming
Organizes business objects and domain logic	Addresses cross-cutting concerns
Example: StudentService	Example: Logging, security and timing

Main idea: OOP organizes business logic; AOP organizes cross-cutting infrastructure logic.

19. Three Fundamental Questions of AOP

WHAT? — What should happen? Example: Logging.
WHERE? — Where should it happen? Example: All service classes and their methods.
WHEN? — When should it happen? Example: At method start and method end.



20. Final Revision Flow



↓

Decorator executes infrastructure logic

↓

Business object executes business logic

↓

AOP provides a structured way to handle cross-cutting concerns

21. Interview Questions and Answers

Q1. What is AOP in Spring?

Answer: AOP is a Spring module that helps separate cross-cutting concerns from business logic.

Q2. What are cross-cutting concerns?

Answer: Concerns such as logging, security, auditing and execution-time measurement that apply across multiple classes and methods.

Q3. Difference between business logic and infrastructure logic?

Answer: Business logic performs domain-specific operations, whereas infrastructure logic provides supporting functionality.

Q4. What is scattering?

Answer: Scattering occurs when the same infrastructure logic is spread across multiple classes.

Q5. What is tangling?

Answer: Tangling occurs when multiple types of logic are mixed inside one method.

Q6. Why are helper functions insufficient?

Answer: Business methods still need explicit calls to helper methods, so infrastructure logic remains mixed with business logic.

Q7. Why is inheritance insufficient?

Answer: Child classes still need to invoke inherited infrastructure methods explicitly.

Q8. What is the Decorator Design Pattern?

Answer: A structural design pattern that wraps an object with another object implementing the same interface to add additional behavior.

Q9. Why does a decorator implement the same interface?

Answer: So it can be used wherever the original business object is expected.

Q10. What is the purpose of @Primary?

Answer: It tells Spring to prefer a particular bean when multiple beans of the same type are available.

Q11. Can we have multiple decorators?

Answer: Yes. Multiple decorators can wrap one another, with each handling a separate supporting concern.

Q12. What three questions does AOP answer?

Answer: What should happen, where should it happen, and when should it happen.

Scope note: These notes cover the lecture's introduction to AOP and its explanation of cross-cutting concerns through the Decorator pattern. The detailed Spring AOP implementation, annotations and advice types are for the subsequent lecture.